Studio Nx

JS 05 — parseLoginHash()

2026-10-07

JS 05 — `parseLoginHash(hash)`

항목	내용
파일	`studionx/frontend/static/js/core/auth.js`
작업	P1-02 프론트 공통 JS · PRD 01 FR-105
작성	Claude (AI) · 2026-10-06 · 사용자 검토
이전 · 다음	04 `loginUrl()` · 06 `isLoggedIn()`

1. 하는 일

주소의 `#login?next=…` 를 읽는다. **모달을 열지(`open`)** 와 **로그인 후 갈 곳(`next`)** 을 돌려준다.

`parseLoginHash('#login?next=%2FmyPage.html')` → `{ open: true, next: '/myPage.html' }`

2. 언제 쓰나

04 `loginUrl()` 은 로그인 주소를 만들고, 이 함수는 `home.html` 에서 그 주소를 읽는다. 두 함수는 같은 주소 형식(`#login?next=…`)을 쓰므로 함께 바꿔야 한다.

상황	부르는 곳
`home.html` 을 열 때	`home.js` `initLoginModalFromHash()` (P1-07) — `open` 이면 로그인 모달을 열고 `next` 를 기억
로그인 성공 후	로그인 폼 처리 (P1-07) — `next` 가 있으면 그리로, 없으면 기본 페이지로

주소	모달	로그인 후
`home.html`	안 열림	—
`home.html#login` (헤더 Sign In · 가입 완료)	열림	기본 페이지
`home.html#login?next=%2FmyPage.html` (`requireLogin`)	열림	`/myPage.html`

3. 코드

import (파일 맨 위)

```
import { api, clearAccessToken, getAccessToken, refreshAccessToken } from './api.js';
```

`auth.js` 맨 위의 import 줄이다. **이 함수는 import 한 것을 쓰지 않는다** — `safeNextPath` 는 같은 파일, `URLSearchParams` · `location` 은 브라우저 기본 기능이다.

함수

```
export function parseLoginHash(hash = location.hash) {
  if (typeof hash !== 'string' || !hash.startsWith('#login')) return { open: false, next: null };
  const rest = hash.slice('#login'.length);
  if (rest && !rest.startsWith('?')) return { open: false, next: null };
  const next = new URLSearchParams(rest.slice(1)).get('next');
```

```
  return { open: true, next: safeNextPath(next) };
}
```

문법 풀이

import 문법	뜻
`import { a, b } from './api.js'`	`api.js` 가 `export` 한 것 중 `a` · `b` 를 이름 그대로 가져옴. `'./'` 는 같은 폴더. 브라우저는 `<script type="module">` 일 때만 `import` 를 읽는다

식	뜻
`hash = location.hash`	기본값 — 지금 주소의 `#` 부터 끝까지 (예: `'#login?next=%2FmyPage.html'`)
`hash.slice('#login'.length)`	앞의 `#login`(6글자)을 잘라낸 나머지 → `''` 또는 `'?next=…'`
`rest && !rest.startsWith('?')`	나머지가 있는데 `?` 로 시작하지 않으면 → `#loginHelp` 같은 **다른** 해시
`rest.slice(1)`	맨 앞 `?` 를 뗀 `next=%2FmyPage.html`
`new URLSearchParams(…).get('next')`	`이름=값&…` 을 읽어 `next` 값을 꺼냄. **인코딩을 자동으로 풀어 줌** — `%2F` → `/`, `%3F` → `?`. 없으면 `null`

hash	rest	결과
`'#login'`	`''`	`{ open: true, next: null }`
`'#login?next=%2FmyPage.html'`	`'?next=%2FmyPage.html'`	`{ open: true, next: '/myPage.html' }`
`'#loginHelp'`	`'Help'`	`{ open: false, next: null }`
`'#features'`	—	`{ open: false, next: null }`

4. 흐름

1. `#login` 으로 시작하지 않으면 → 열지 않음.
2. `#login` 뒤가 비었거나 `?` 로 시작하는지 확인 (`#loginHelp` 거름).
3. `?` 뒤에서 `next` 를 꺼낸다 (인코딩 풀림).
4. `safeNextPath` 로 검사 — 위험하면 `next: null` 이지만 **모달은 연다**.

5. 왜 이렇게

선택	이유
해시(`#`)에 둠	서버로 안 감 · 페이지를 다시 불러오지 않음 (04 보안)
`URLSearchParams`	`&` · `=` · `%XX` 를 직접 자르면 실수하기 쉬움. 브라우저 표준 방식으로 읽음
읽을 때도 `safeNextPath`	주소는 누구나 바꿀 수 있음 — **만들 때 검사했어도 읽을 때 다시**
위험한 `next` 여도 `open: true`	로그인 자체는 정상. 갈 곳만 버리고 기본 페이지로
결과를 객체 하나로	부르는 쪽이 `const { open, next } = parseLoginHash()` 한 줄로 받음

6. 시험

입력	결과
`#login`	열림 · `next` 없음
`#login?next=%2FmyPage.html`	열림 · `/myPage.html`
`#login?next=%2FstudioNx.html%3FprojectId%3D3`	열림 · `/studioNx.html?projectId=3`
`#login?next=%2FmyProject.html%3Fq%3Da%26page%3D2`	열림 · `/myProject.html?q=a&page=2`
`#login?next=https%3A%2F%2Fevil.com` · `…%2F%2Fevil.com`	열림 · `next: null` (풀면 `https://…` · `//evil.com` → 거부)
`#login?foo=1`	열림 · `next: null`
`#loginHelp` · `#features` · `''`	안 열림
인자 없음 (`location.hash` 사용)	`location.hash` 를 읽음
`loginUrl()` 로 만든 주소 → 이 함수로 읽기	원래 경로 그대로 (왕복)